

Bayesian Inference of F1 Driver Ability

STAT 405: Bayesian Statistics - Sohbat Sandhu (79661179)

18 April 2026

1 Introduction

Formula One (F1) performance is shaped by a complex interaction between driver decision-making, car performance, tire degradation, and race conditions. Modern telemetry makes these interactions measurable at high resolution: instead of analyzing only lap time, we can study micro-sector time, where a lap is segmented into 100 equal-distance regions. A central challenge is that observed micro-sector times are not direct measurements of driver skill. They confound skill with team/car advantages, tire compound and wear, temperature, and race interruptions.

1.1 Reasearch Question

Using microsectors telemetry and lap summary data from the **2023 Abu Dhabi Grand Prix**, we aim to infer latent driver ability and rank drivers with uncertainty while controlling for confounders and separaing driver effects from (1) team/car effects and (2) systematic track-geometry differences across microsectors. A bayesian hierarchical model is appropriate for this task as each microsector time is a noisy measurement and is influenced by multiple hierarchical factors with uneven sample sizes (influenced by driver choices like choosing different tire compounds and using different strategies).

1.2 Literature Review

Van Kesteren and Bergkamp (2023) developed a Bayesian multilevel rank-ordered logit model using *race finishing positions over multiple races and seasons* to infer driver skill and constructor advantage, showing that constructor effects dominate variation in outcomes. Bell et al. (2016) apply a cross-classified multilevel model to historical race results, decomposing performance into driver and team components. More recently, Cappello and Hoegh (2025) introduce a Bayesian state-space model to capture latent tire degradation and its impact on lap times, demonstrating the importance of dynamic latent processes in explaining performance variation. While these studies establish strong foundations, they primarily rely on coarse outcome variables such as finishing positions or lap times, or focus on single-driver dynamics. In contrast, this project differs by modelling micro-sector level telemetry, allowing inference on driver ability at a much finer spatial resolution.

2 Preliminary Analysis

We start with a preliminary analysis of the lap time data for 26 clean laps across 20 drivers (2 per team).

Table 1: *Team-Driver Lap Time Stats for 26 Clean laps (in s)*

Team	D1	Mean	SD	Min.	q25, q75	D2	Mean	SD	Min.	q25, q75
RedBull	VER	88.932	1.153	86.993	87.91-89.57	PER	89.163	1.279	87.758	87.984-89.7
Mercedes	HAM	89.656	1.05	88.372	88.81-89.9	RUS	89.526	1.019	88.187	88.69-90.2
Ferrari	LEC	89.331	0.937	88.221	88.55-90.0	SAI	90.522	0.766	89.635	89.85-91.09
McLaren	NOR	89.467	0.942	88.164	88.60-89.87	SAI	89.731	1.172	88.149	88.74-90.59
AstonMartin	ALO	89.742	1.081	88.491	88.62-90.6	STR	89.891	1.129	88.177	88.86-90.85
Alpine	GAS	90.212	0.738	89.307	89.52-90.7	OCO	90.656	0.553	90.113	90.26-90.9

Continued on next page

Table 1 – continued from previous page

Team	D1	Mean	SD	Min.	q25, q75	D2	Mean	SD	Min.	q25, q75
Williams	ALB	90.083	1.442	87.845	88.75-90.7	SAR	90.328	1.338	88.64	88.95-91.4
AlphaTauri	TSU	90.241	0.816	89.256	89.47-90.9	RIC	89.947	1.213	88.571	88.77-90.7
AlfaRomeo	BOT	91.41	0.885	89.998	90.57-92.0	ZHO	90.524	1.030	88.951	89.7-91.2
Haas	MAG	90.968	0.763	89.934	90.44-91.5	SAI	90.689	0.909	89.456	90.09-91.4

Table 1 observes lap-level performance by driver. We observe that while there are differences in mean lap times across drivers and teams, there is also considerable overlap in their distributions, highlighting the need for a more nuanced analysis to disentangle driver ability from other factors.

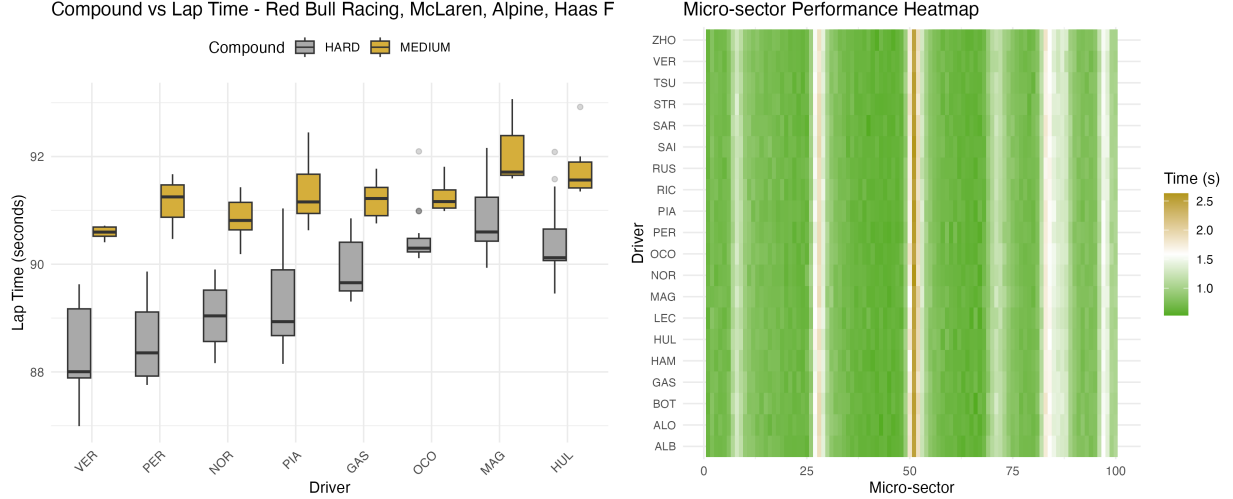


Figure 1.1: Lap time distributions by tire compound and team (every 2 drivers are a single team); Figure 1.2: Heatmap of micro-sector performance across drivers and sectors.

Figure 1.1 shows that lap times are faster with harder compounds, despite comparatively softer compounds generally yielding faster times. This suggests other factors like track temperature, tire degradation and stint length may be influencing performance. We also see an increase trend in lap times across teams, denoting that teams have different performance thresholds. Figure 1.2 reveals patterns in micro-sector performance, with consistent time increase corresponding to turns indicating the importance of reflecting track geometry. For example, sectors 26-31, 49-55, and 80-88 correspond to turns 5, 6-7, and 12-14 respectively (see Figure x.x), and show variability across drivers (hard to distinguish).

3 Methodology

3.1 Modelling

We model micro-sector and lap time based on logarithm scale to ensure predictions remain positive on the original scale and to improve approximate normality. We build two models: a lap level baseline (naive) model and a full hierarchical model (micro-sector level) for driver ranking and performance decomposition. All variables are specified from Sections 3.1.1-3.1.3.

3.1.1 Naive Model (Lap-Level Inferencing)

Let $y_i = \log(\text{LapTimeSeconds}_i)$ for each observation i , we model $y_i \mid \mu_i, \sigma \sim \mathcal{N}(\mu_i, \sigma^2)$ where the performance is based on:

$$\mu_i = \alpha_0 + \alpha_{d[i]} + \beta_{c[i]} + \beta_{\text{tyre}} \text{TyreLife}_i + \beta_{\text{temp}} \text{TrackTemp}_i + \beta_{\text{status}} \text{TrackStatus}_i$$

Driver ability is included via a non-centered parameterization: $\alpha_d = \sigma_{\text{driver}} z_d$, $z_d \sim \mathcal{N}(0, 1)$. Tyre compound effects are centered using β_c . For other priors we use $\beta_{\text{tyre}}, \beta_{\text{temp}}, \beta_{\text{status}} \sim \mathcal{N}(0, 0.5)$. This model provides a baseline driver ranking adjusted for tyre degradation (based on usage, weather and compound) and acceptable racing conditions, without separating team/car contribution.

3.1.2 Main Hierarchical Model (Micro-sector Level Inferencing)

Let $y_i = \log(\text{TimeSeconds}_i)$ for each micro-sector observation i . We use a robust likelihood, $y_i \mid \mu_i, \sigma, \nu \sim t_\nu(\mu_i, \sigma)$ where the performance is based on:

$$\mu_i = \alpha_0 + \alpha_{d[i]} + \gamma_{t[i]} + \delta_{m[i]} + \theta_i + \beta_X^\top X_i + \beta_{\text{status}} \text{TrackStatus}_i.$$

Hierarchical effects with $z_d, z_t, z_m \sim \mathcal{N}(0, 1)$:

$$\alpha_d = \sigma_{\text{driver}} z_d \text{ (Driver Effects)}, \gamma_t = \sigma_{\text{team}} z_t \text{ (Team Effects)}, \delta_m = \sigma_{\text{micro}} z_m \text{ (Micro-sector Effects)}$$

Tyre degradation is accounted with compound effects (β_c), and the following track and tyre usage conditions:

$$\theta_i = \beta_c[i] + \beta_{\text{tyre}} \text{TyreLife}_i + \beta_{\text{temp}} \text{TrackTemp}_i + \beta_{\text{inter}} (\text{TyreLife}_i \cdot \text{TrackTemp}_i)$$

With regression priors as $\beta_{\text{tyre}}, \beta_{\text{temp}}, \beta_{\text{inter}}, \beta_X, \beta_{\text{status}} \sim \mathcal{N}(0, 0.5)$ This hierarchical model decomposes performance into driver ability, team/car effects, and track geometry, while accounting for tyre–environment interactions and driver inputs, enabling a more granular and fair estimation of latent driver skill.

3.1.3 Prior Choice Motivations

All continuous covariates (e.g., TyreLife, TrackTemp, driver inputs) are standardized, allowing the use of moderately informative priors with consistent interpretation across parameters. The intercept is assigned a weakly informative prior, $\alpha_0 \sim \mathcal{N}(0, 2)$, for baseline log-time variation without permitting extreme values. Scale parameters are given exponential priors, $\sigma, \sigma_{\text{driver}}, \sigma_{\text{team}}, \sigma_{\text{micro}} \sim \text{Exp}(1)$, to ensure positive values. Hierarchical effects using $z \sim \mathcal{N}(0, 1)$, $\alpha = \sigma z$, improves sampling stability in high-dimensional spaces. Due to standardization, regression coefficients for tyre effects, environmental variables, and driver inputs use $\beta \sim \mathcal{N}(0, 0.5)$, to indicate moderate effect on log-time variable by 1SD. Categorical tyre compound effects are centered, $\beta_c = \beta_c^{\text{raw}} - \bar{\beta}^{\text{raw}}$, $\beta_c^{\text{raw}} \sim \mathcal{N}(0, 0.5)$ to ensure performance is relative to the global average across all compound. The df parameter for the Student- t likelihood is assigned $\nu \sim \mathcal{U}(2, 60)$ to allow robustness to outliers which can be due to minor errors. Overall, these priors are numerous and robust to leveraging information for decent inferences.

4 Results & Interpretations

Posterior inference is performed using (i) NUTS MCMC via Stan (cmdstanr) as the reference method and (ii) mean-field variational inference (VI) as a fast approximation.

4.1 Diagnostics

4.2 Variational Inference

5 Discussion

5.1 Limitations

5.2 Conclusions

6 Appendix

6.1 GitHub

Outputs and code available at [GitHub Repository](#)

NOTE: To ensure no issues in running the code for reproducing the outputs and analysis follow the instructions to setup your environment and run instructions in the `README.md` file.

6.2 Stan Implementation

6.2.1 Naive Model (lap-level inferencing)

```
data {  
  int<lower=1> N;  
  int<lower=1> D;  
  int<lower=1> C;  
  
  array[N] int<lower=1,upper=D> driver;  
  array[N] int<lower=1,upper=C> compound;  
  array[N] int<lower=0,upper=1> TrackStatus;  
  
  vector[N] y; // log_lap_time  
  
  vector[N] TyreLife_z;  
  vector[N] TrackTemp_z;  
}  
  
parameters {  
  real alpha0;  
  
  real<lower=1e-6> sigma_driver;  
  vector[D] z_driver;  
  
  vector[C] beta_comp_raw;  
  
  real beta_tyre;  
  real beta_temp;  
  
  real beta_status;  
  
  real<lower=1e-6> sigma;  
}  
  
transformed parameters {  
  vector[D] alpha_driver = sigma_driver * z_driver;  
  vector[C] beta_comp = beta_comp_raw - mean(beta_comp_raw);  
}  
  
model {  
  alpha0 ~ normal(0, 2);  
  
  // Use exponential to avoid piling mass at ~0, and keep sigma > 0.
```

```

sigma ~ exponential(1);
sigma_driver ~ exponential(1);

z_driver ~ std_normal();

beta_comp_raw ~ normal(0, 0.5);

beta_tyre ~ normal(0, 0.5);
beta_temp ~ normal(0, 0.5);

beta_status ~ normal(0, 0.5);

for (n in 1:N) {
  real mu = alpha0
    + alpha_driver[driver[n]]
    + beta_comp[compound[n]]
    + beta_tyre * TyreLife_z[n]
    + beta_temp * TrackTemp_z[n]
    + beta_status * TrackStatus[n];

  y[n] ~ normal(mu, sigma);
}

generated quantities {
  vector[N] log_lik;
  for (n in 1:N) {
    real mu = alpha0
      + alpha_driver[driver[n]]
      + beta_comp[compound[n]]
      + beta_tyre * TyreLife_z[n]
      + beta_temp * TrackTemp_z[n]
      + beta_status * TrackStatus[n];

    log_lik[n] = normal_lpdf(y[n] | mu, sigma);
  }
}

```

6.2.2 Hierarchical Model (micro-sector level inferencing)

```

data {
  int<lower=1> N;
  int<lower=1> D;
  int<lower=1> T;
  int<lower=1> M;
  int<lower=1> C;

  array[N] int<lower=1,upper=D> driver;
  array[N] int<lower=1,upper=T> team;
  array[N] int<lower=1,upper=M> micro;
  array[N] int<lower=1,upper=C> compound;
}

```

```

array[N] int<lower=0,upper=1> TrackStatus;

vector[N] y;

vector[N] TyreLife_z;
vector[N] TrackTemp_z;
vector[N] TyreLife_x_TrackTemp;

int<lower=1> K;
matrix[N, K] X;

int<lower=1> ppc_n;
array[ppc_n] int<lower=1,upper=N> ppc_idx;
}

parameters {
  real alpha0;

  real<lower=1e-6> sigma_driver;
  vector[D] z_driver;

  real<lower=1e-6> sigma_team;
  vector[T] z_team;

  real<lower=1e-6> sigma_micro;
  vector[M] z_micro;

  vector[C] beta_comp_raw;

  real beta_tyre;
  real beta_temp;
  real beta_inter;

  vector[K] beta_X;
  real beta_status;

  real<lower=1e-6> sigma;
  real<lower=2, upper=60> nu;
}

transformed parameters {
  vector[D] alpha_driver = sigma_driver * z_driver;
  vector[T] gamma_team = sigma_team * z_team;
  vector[M] delta_micro = sigma_micro * z_micro;

  vector[C] beta_comp = beta_comp_raw - mean(beta_comp_raw);
}

model {
  alpha0 ~ normal(0, 2);

  // scale priors (more stable than half-normal at exactly 0)
  sigma ~ exponential(1);

```

```

sigma_driver ~ exponential(1);
sigma_team ~ exponential(1);
sigma_micro ~ exponential(1);

z_driver ~ std_normal();
z_team ~ std_normal();
z_micro ~ std_normal();

beta_comp_raw ~ normal(0, 0.5);

beta_tyre ~ normal(0, 0.5);
beta_temp ~ normal(0, 0.5);
beta_inter ~ normal(0, 0.5);

beta_X ~ normal(0, 0.5);
beta_status ~ normal(0, 0.5);

nu ~ uniform(2, 60);

for (n in 1:N) {
  real mu = alpha0
    + alpha_driver[driver[n]]
    + gamma_team[team[n]]
    + delta_micro[micro[n]]
    + beta_comp[compound[n]]
    + beta_tyre * TyreLife_z[n]
    + beta_temp * TrackTemp_z[n]
    + beta_inter * TyreLife_x_TrackTemp[n]
    + dot_product(beta_X, X[n])
    + beta_status * TrackStatus[n];

  y[n] ~ student_t(nu, mu, sigma);
}

generated quantities {
  vector[N] log_lik;
  vector[ppc_n] y_rep;

  real driver_frac = square(sigma_driver) / (square(sigma_driver) + square(sigma_team));
  real team_frac = 1 - driver_frac;

  for (n in 1:N) {
    real mu = alpha0
      + alpha_driver[driver[n]]
      + gamma_team[team[n]]
      + delta_micro[micro[n]]
      + beta_comp[compound[n]]
      + beta_tyre * TyreLife_z[n]
      + beta_temp * TrackTemp_z[n]
      + beta_inter * TyreLife_x_TrackTemp[n]
      + dot_product(beta_X, X[n])
      + beta_status * TrackStatus[n];

```

```

    log_lik[n] = student_t_lpdf(y[n] | nu, mu, sigma);
}

for (j in 1:ppc_n) {
    int n = ppc_idx[j];
    real mu = alpha0
        + alpha_driver[driver[n]]
        + gamma_team[team[n]]
        + delta_micro[micro[n]]
        + beta_comp[compound[n]]
        + beta_tyre * TyreLife_z[n]
        + beta_temp * TrackTemp_z[n]
        + beta_inter * TyreLife_x_TrackTemp[n]
        + dot_product(beta_X, X[n])
        + beta_status * TrackStatus[n];

    y_rep[j] = student_t_rng(nu, mu, sigma);
}
}

```

6.3 Data Information

Variable	Type	Description
Car Telemetry Data across 100 Micro-sectors		
Driver	Categorical	Three-letter driver abbreviation (e.g., VER, HAM)
Team	Categorical	The team or constructor name (e.g., Red Bulls Racing, Mercedes, Ferrari)
LapNumber	Integer (1–58)	Sequential lap index during the race (Race for this session 58 laps long)
micro_sector	Integer (0–99)	Track divided into 100 equal-distance segments
Speed	Continuous (km/h)	Instantaneous vehicle speed at given micro-sector
Throttle	Continuous (0–100)	Throttle application percentage
Brake	Continuous (0–100)	Brake application percentage
RPM	Continuous	Engine revolutions per minute
TyreLife	Integer	Number of laps completed on current tire set
AirTemp	Continuous (°C)	Ambient air temperature
TrackTemp	Continuous (°C)	Track temperature
LapTimeSeconds	Continuous (s)	Total lap time
TimeSeconds	Continuous (s)	Time elapsed from the start of the current micro-sector to its end
Lap-Level Driver Summary Data for 26 Clean Laps		
Driver	Categorical	Three-letter driver abbreviation (e.g., VER, HAM)
Team	Categorical	The team or constructor name (e.g., Red Bulls Racing, Mercedes, Ferrari)

LapNumber	Integer (1–58)	Sequential lap index during the race (Race is 58 long)
LapTimeSeconds	Continuous (s)	Total lap time
Stint	Integer	Stint index between pit stops
Compound	Categorical	Tire compound (SOFT, MEDIUM, HARD)
TrackStatus	Binary	Race continuation status (1=Green, others=flags)
PitInTime	Date-time or NA	Timestamp of pit entry
PitOutTime	Date-time or NA	Timestamp of pit exit

Table 2: *Table Summary of Variables in the Processed Telemetry for 26 Clean Laps*

6.3.1 Processed Datasets

The following are the previews of the two main datasets used in the analysis. The first dataset contains micro-sector telemetry data for 100 segments of each lap, while the second dataset summarizes lap-level information for 26 clean laps.

```
## Rows: 520
## Columns: 12
## $ Driver      <chr> "VER", "VER", "VER", "VER", "VER", "VER", "VER", "VER", ~
## $ driver_id   <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
## $ Team        <chr> "Red Bull Racing", "Red Bull Racing", "Red Bull Racing"~
## $ team_id     <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
## $ LapNumber   <dbl> 2, 3, 4, 9, 10, 11, 19, 20, 21, 25, 26, 27, 28, 39, 40, ~
## $ Compound    <chr> "MEDIUM", "MEDIUM", "MEDIUM", "MEDIUM", "MEDIUM", "MEDI~
## $ compound_id <dbl> 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, 3, ~
## $ TyreLife     <dbl> 1, 2, 3, 8, 9, 10, 2, 3, 4, 8, 9, 10, 11, 22, 23, 1, 2, ~
## $ Stint        <dbl> 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 3, 3, 3, 3, ~
## $ TrackStatus <dbl> 0, 1, 0, 1, 1, 1, 0, 0, 1, 0, 0, 0, 0, 1, 0, 1, 1, 1, ~
## $ LapTimeSeconds <dbl> 90.710, 90.408, 90.716, 90.632, 90.506, 90.561, 89.386, ~
## $ log_lap_time <dbl> 4.507668, 4.504333, 4.507734, 4.506807, 4.505416, 4.506~
```

Table x.x: *Car Telemetry Data across 100 Micro-sectors driven by 20 drivers in 2023 F1 Abu Dhabi Grand Prix*

```
## Rows: 52,000
## Columns: 19
## $ Driver      <chr> "VER", "VER", "VER", "VER", "VER", "VER", "VER", "VER", ~
## $ driver_id   <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
## $ Team        <chr> "Red Bull Racing", "Red Bull Racing", "Red Bull Racing"~
## $ team_id     <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
## $ compound_id <dbl> 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, ~
## $ LapNumber   <dbl> 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, ~
## $ micro_sector <dbl> 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, ~
## $ Speed       <dbl> 233.0000, 248.7500, 262.0000, 268.0000, 271.7500, 245.6~
## $ Throttle    <dbl> 100.00000, 100.00000, 100.00000, 100.00000, 86.75000, 0~
## $ Brake       <dbl> 0, 0, 0, 0, 0, 100, 100, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, ~
## $ rpm         <dbl> 11528.000, 11143.000, 11075.500, 10731.500, 10698.000, ~
## $ AirTemp     <dbl> 27, 27, 27, 27, 27, 27, 27, 27, 27, 27, 27, 27, 27, 27, ~
## $ TrackTemp   <dbl> 33.6, 33.6, 33.6, 33.6, 33.6, 33.6, 33.6, 33.6, 33.6, 3~
## $ TyreLife     <dbl> 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, ~
```



```
## variable      mean      q5      q95  rhat  ess_bulk  ess_tail
## <chr>         <dbl>   <dbl>   <dbl> <dbl>    <dbl>    <dbl>
## 1 sigma_driver 0.006  0.004  0.008 1.01     644.     742.
## 2 beta_tyre    0.005  0.005  0.006 1        3990.    3468.
## 3 beta_temp    0.011  0.01   0.012 1        4393.    3451.
## 4 beta_status -0.002 -0.003 -0.001 1        4756.    2540.
## 5 sigma        0.006  0.005  0.006 1        3348.    2381.
```

Table x.x: Summary of key diagnostics for the MCMC naive model, including R -hat values, effective sample sizes, and posterior means for key parameters.

```
## # A tibble: 20 x 7
## variable      mean      q5      q95  rhat  ess_bulk  ess_tail
## <chr>         <dbl>   <dbl>   <dbl> <dbl>    <dbl>    <dbl>
## 1 alpha_driver[1] -0.009 -0.012 -0.006 1.00     533.    1048.
## 2 alpha_driver[2] -0.007 -0.01   -0.004 1.00     513.    1050.
## 3 alpha_driver[3] -0.003 -0.006  0      1.00     554.    1141.
## 4 alpha_driver[4] -0.005 -0.008 -0.002 1.00     574.    1040.
## 5 alpha_driver[5] -0.006 -0.009 -0.003 1.00     509.    1031.
## 6 alpha_driver[6]  0.003  0      0.006 1.00     588.    1393.
## 7 alpha_driver[7] -0.006 -0.009 -0.003 1.00     598.    1220.
## 8 alpha_driver[8] -0.003 -0.006  0      1.00     541.    1143.
## 9 alpha_driver[9] -0.002 -0.005  0.001 1.00     511.     943.
## 10 alpha_driver[10] -0.001 -0.004  0.002 1.00     527.    1077.
## 11 alpha_driver[11]  0.002 -0.001  0.006 1.00     585.    1287.
## 12 alpha_driver[12]  0.001 -0.002  0.003 1.00     566.    1145.
## 13 alpha_driver[13]  0.003  0      0.006 1.00     539.    1150.
## 14 alpha_driver[14]  0.005  0.002  0.008 1.00     537.    1175.
## 15 alpha_driver[15] -0.001 -0.004  0.002 1.00     591.    1289.
## 16 alpha_driver[16] -0.002 -0.005  0.001 1.00     549.    1227.
## 17 alpha_driver[17]  0.01   0.007  0.013 1.00     535.    1064.
## 18 alpha_driver[18]  0.005  0.002  0.008 1.00     549.    1050.
## 19 alpha_driver[19]  0.008  0.005  0.011 1.00     512.    1086.
## 20 alpha_driver[20]  0.007  0.004  0.01   1.00     524.    1228.
```

Table x.x: Summary of driver ability diagnostics for the MCMC naive model, including R -hat values, effective sample sizes, and posterior means for key parameters

7 References

1. Source: Open F1 - The open source API for Formula 1 data <https://openf1.org/>
2. Source: Collected Primary Datasets - https://github.com/sohbatSandhu/f1_telemetry_bayesian_analysis/tree/main/data/main
3. Source: Backup Datasets (1) Kaggle: <https://www.kaggle.com/datasets/gixarde31/f1-2023-abu-dhabi-gp-telemetry-and-lap-data> (2) GitHub repository: https://github.com/sohbatSandhu/f1_telemetry_bayesian_analysis/tree/main/data/backup
4. van Kesteren EJ, Bergkamp T. Bayesian analysis of Formula One race results: disentangling driver skill and constructor advantage. J Quant Anal Sports. 2023 Jul 25;19(4):273-293. doi: 10.1515/jqas-2022-0021. PMID: 38020583; PMCID: PMC10660124.
5. Bell, Andrew & Smith, James & Sabel, Clive & Jones, Kelvyn. (2016). Formula for success: Multilevel modelling of Formula One Driver and Constructor performance, 1950-2014. Journal of Quantitative Analysis in Sports. 12. 99-112. 10.1515/jqas-2015-0050.

6. Cappello, C., & Hoegh, A. (2025). A state-space approach to modeling tire degradation in Formula 1 racing. arXiv:2512.00640. <https://doi.org/10.48550/arXiv.2512.00640>